

Time series Modelling Using Graph Neural Network

Whether to help a better forecasting power

03.29. 2022

Copyright©. Qraft Technologies. All Rights Reserved.

QRAFT

Contents

I Introduction

II What is GCN

III Experiment

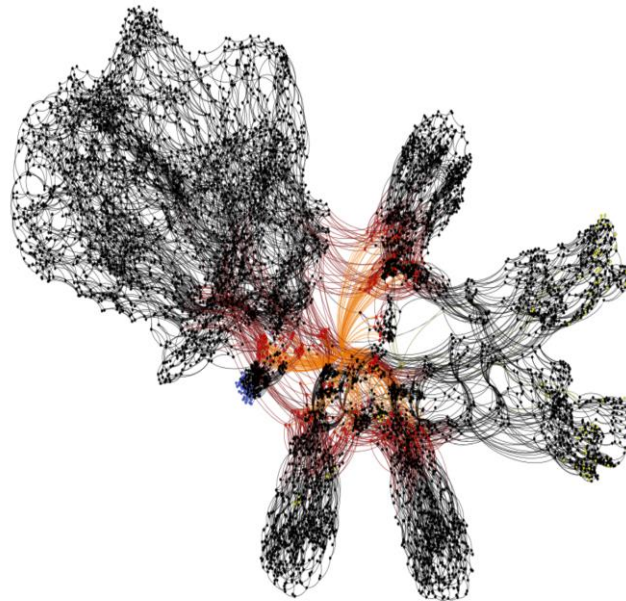
IV Future work

| Introduction

I. Introduction

Definition

GNN are deep learning models aiming at addressing graph-related tasks in an end-to-end manner.[1]

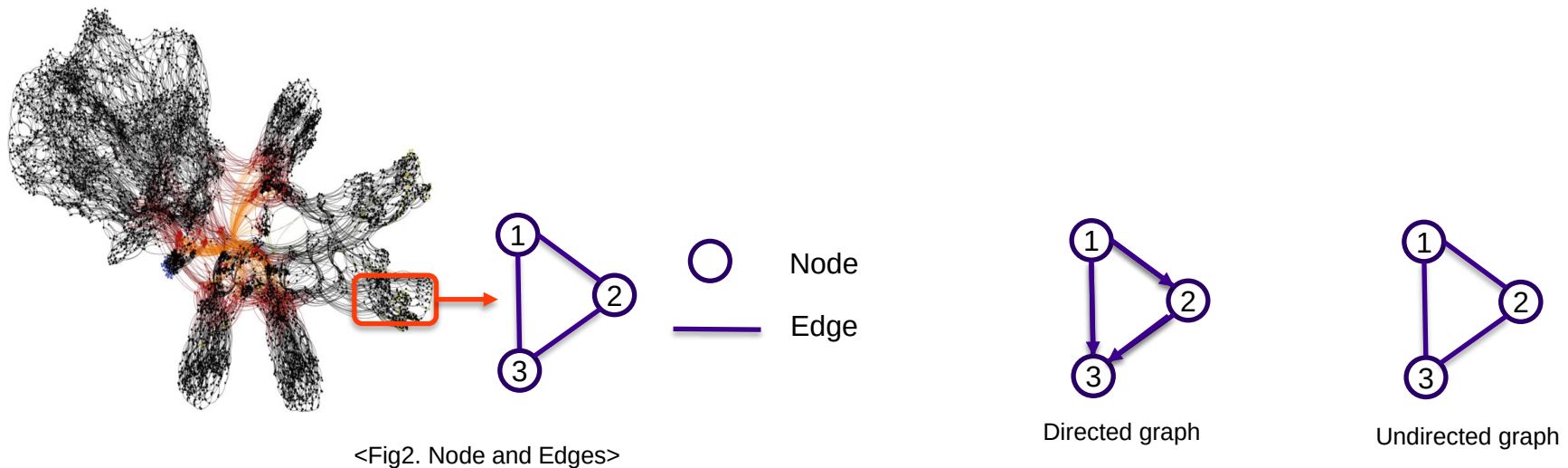


<Fig1. Example of a giant graph>

I. Introduction

What is Graph

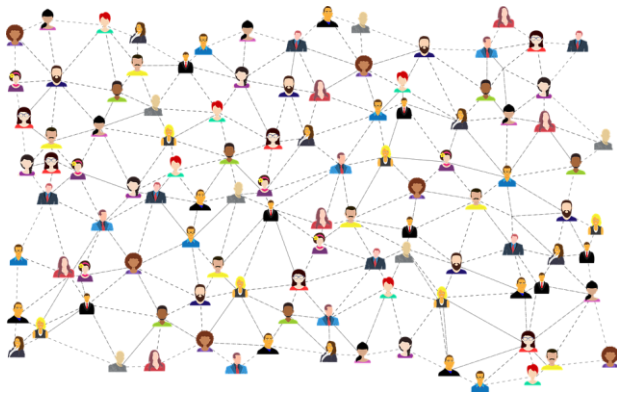
- Graph is a data structure consisting of two components: *Nodes(Vertices)*, *Edges*.
- Graph shows **Relationships**(Connectivity)
- Represented as $G = (V, E)$
 - V is a set of nodes
 - E is the edges between nodes



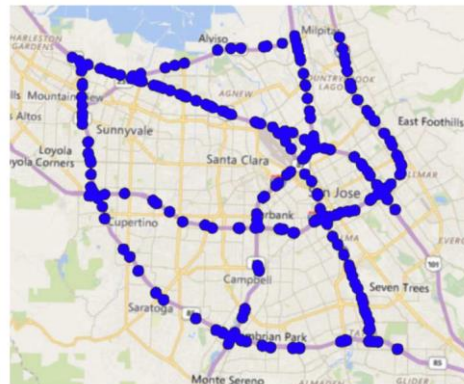
I. Introduction

Graph Example

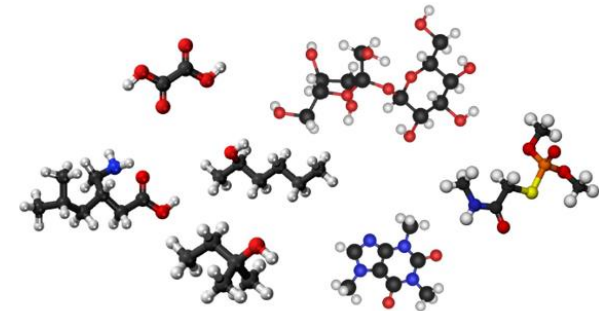
Social network, Traffic, Molecules, Citation, etc...



Social Network



Traffic



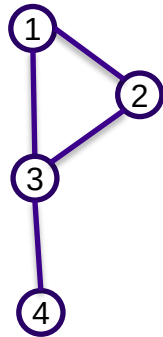
Molecules

<Fig3. Graph examples>

I. Introduction

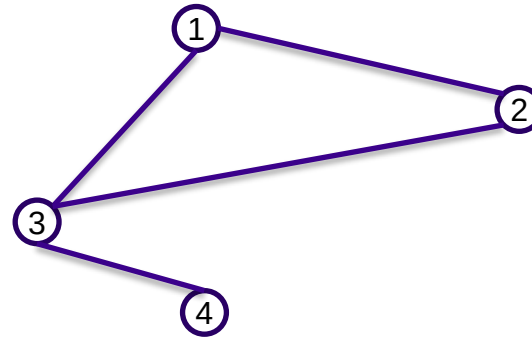
Why is it hard to analyze a Graph

1. Often not exist in a Euclidean space
 - Cannot be represented by coordinate systems
2. No fixed form



Graph A

=



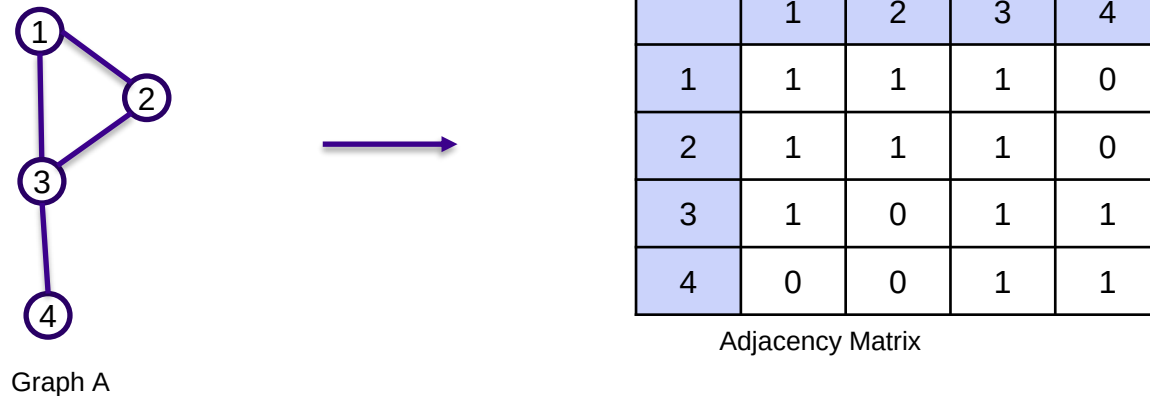
Graph B

I. Introduction

How to represent a Graph?

Adjacency Matrix – shows connectivity

Network structure is realized as adjacency matrix denoted by $A \in R^{N \times N}$, where N is the number of nodes.



<Fig4. Adjacency matrix example>

|| What is GCN

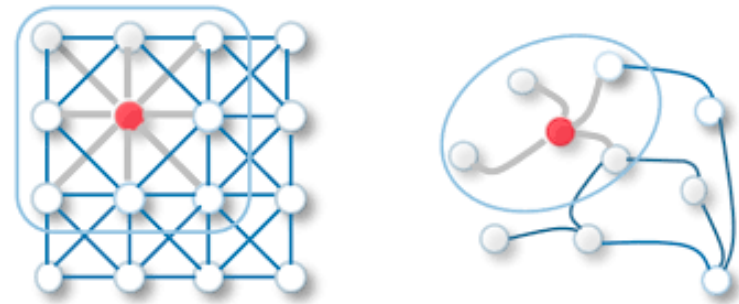
II. What is GCN

What is GNN

- ***GNN*** are deep learning models aiming at addressing graph-related tasks in an end-to-end manner.[1]
- GNN is to determine the “node embedding” of each node, by referring the information on its neighboring nodes.

II. What is GCN (Graph Convolution Network)

- Convolution on Graph data structure
 - Number of nodes connections vary
 - Nodes are unordered
- An essential operation to extract a node's features by aggregating neighboring node's feature.
 - Its filter is Adjacency matrix
- Basic GCN layer is defined as:



<Fig5. 2D convolution vs Graph Convolution>

$$z = AXW$$

where,

$A \in R^{N \times N}$: adjacency matrix

$X \in R^{N \times D}$: input signals

$W \in R^{D \times M}$: model parameter matrix

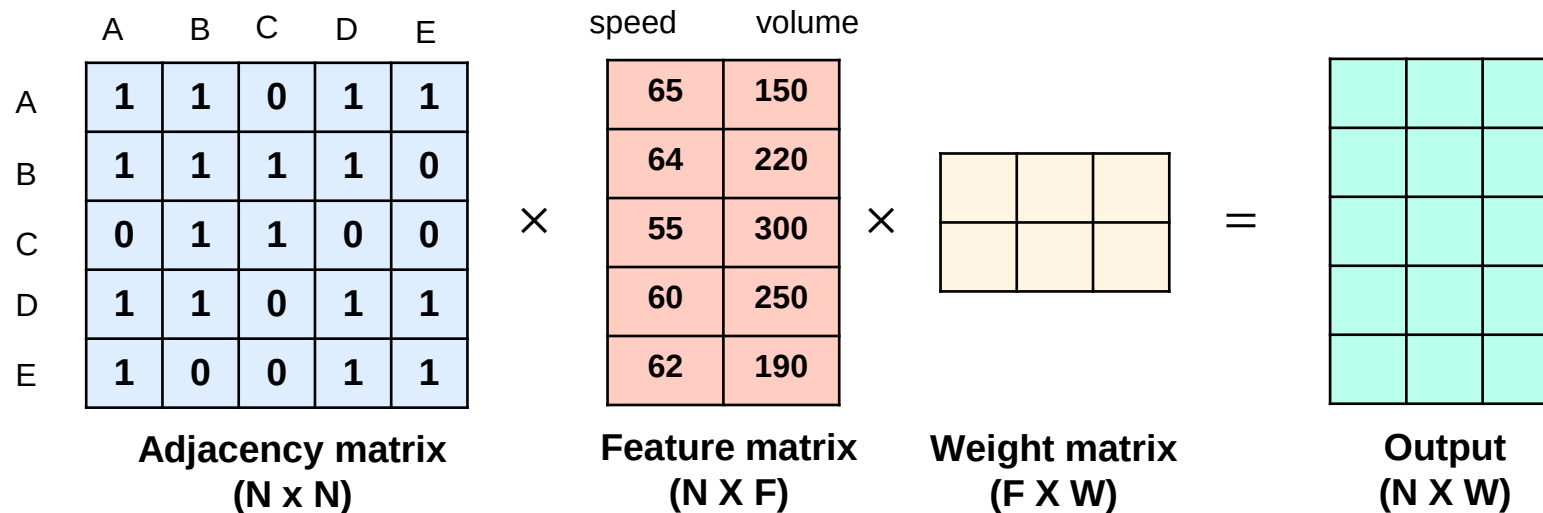
$Z \in R^{N \times M}$: output

II. What is GCN (Graph Convolution Network)

How GCN operate?

Graph Convolution layer: $Z = AXW$

Example: Traffic signal

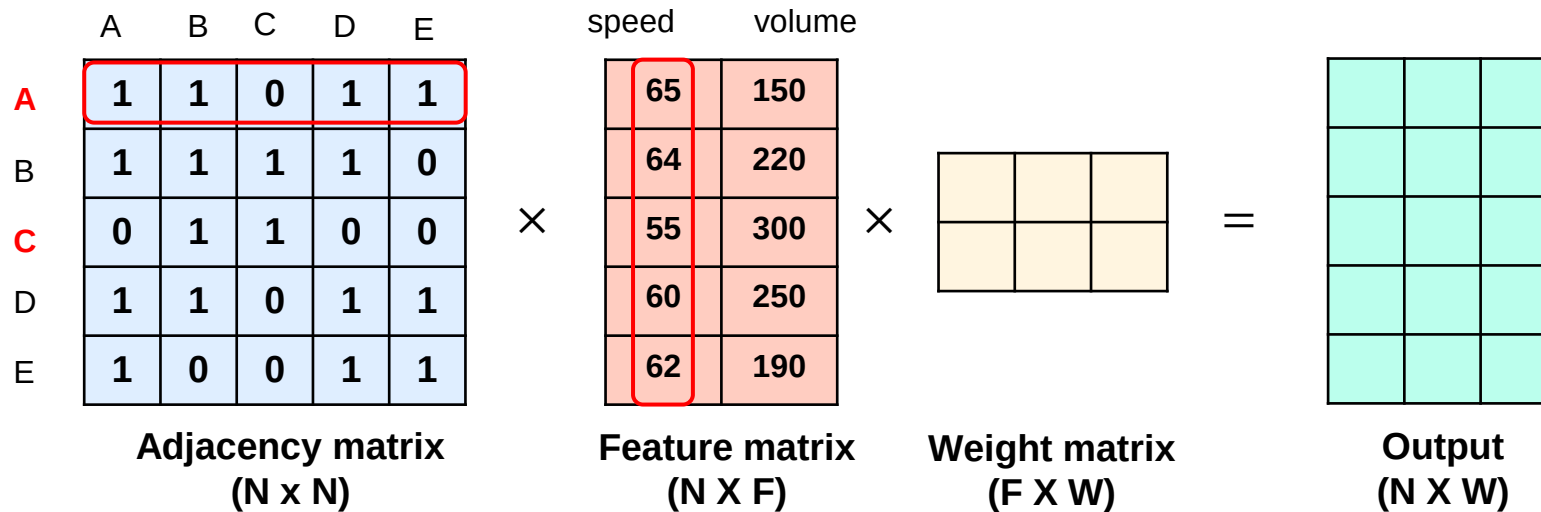


II. What is GCN (Graph Convolution Network)

Needs for normalizing adjacency matrix

- To have numerical stability

Example: Traffic signal



'a' node matrix summation :
 $65 + 64 + 60 + 62 = 251$

'c' node matrix summation :
 $64 + 55 = 119$

Not fair

II. What is GCN (Graph Convolution Network)

Needs for normalizing adjacency matrix

$\tilde{A}$: Normalized adjacency matrix

- $\tilde{A}$ normalized adjacency matrix
 - An unique representation that combines the degree information of each node and their adjacency information in the graph.
 - To train feature value for all nodes with fair condition
- Representation

$$\tilde{A} = D^{-1/2} A D^{-1/2}$$

$A \in R^{N \times N}$: adjacency matrix

$D \in R^{N \times N}$: degree matrix

$$D_{i,j} := \begin{cases} \deg(v_i) & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

$\deg(v_i)$: degree of a node

A
adjacency matrix

1	1	1	0
1	1	1	1
1	1	1	0
0	1	0	1

D
degree matrix

3	0	0	0
0	4	0	0
0	0	3	0
0	0	0	2

II. What is GCN (Graph Convolution Network)

Needs for normalizing adjacency matrix

$\tilde{A}$: Normalized adjacency matrix

- $\tilde{A}$ normalized adjacency matrix

$$\tilde{A} = D^{-1/2} A D^{-1/2}$$

A adjacency matrix	D degree matrix	$D^{-1/2}$ degree matrix																																																
<table><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td></tr></table>	1	1	1	0	1	1	1	1	1	1	1	0	0	1	0	1	<table><tr><td>3</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>4</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>3</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>2</td></tr></table>	3	0	0	0	0	4	0	0	0	0	3	0	0	0	0	2	<table><tr><td>$\frac{1}{\sqrt{3}}$</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>$\frac{1}{\sqrt{4}}$</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>$\frac{1}{\sqrt{3}}$</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>$\frac{1}{\sqrt{2}}$</td></tr></table>	$\frac{1}{\sqrt{3}}$	0	0	0	0	$\frac{1}{\sqrt{4}}$	0	0	0	0	$\frac{1}{\sqrt{3}}$	0	0	0	0	$\frac{1}{\sqrt{2}}$
1	1	1	0																																															
1	1	1	1																																															
1	1	1	0																																															
0	1	0	1																																															
3	0	0	0																																															
0	4	0	0																																															
0	0	3	0																																															
0	0	0	2																																															
$\frac{1}{\sqrt{3}}$	0	0	0																																															
0	$\frac{1}{\sqrt{4}}$	0	0																																															
0	0	$\frac{1}{\sqrt{3}}$	0																																															
0	0	0	$\frac{1}{\sqrt{2}}$																																															

II. What is GCN (Graph Convolution Network)

Needs for normalizing adjacency matrix

$\tilde{A}$: Normalized adjacency matrix

- $\tilde{A}$ normalized adjacency matrix

$$\tilde{A} = D^{-1/2} A D^{-1/2}$$

The diagram illustrates the normalization of the adjacency matrix A to form the normalized adjacency matrix $\tilde{A}$ using the formula $\tilde{A} = D^{-1/2} A D^{-1/2}$.

The matrix $D^{-1/2}$ (green) is row-normalized, with the first row highlighted by a red box. The matrix A (blue) is the adjacency matrix, with the first column highlighted by a red box and the first row highlighted by a blue box. The matrix $D^{-1/2}$ (green) is column-normalized, with the first column highlighted by a blue box.

The resulting matrix $\tilde{A}$ is the normalized adjacency matrix, which is the product of the row-normalized $D^{-1/2}$, the adjacency matrix A , and the column-normalized $D^{-1/2}$.

Labels below the matrices indicate the normalization process:

- row-normalised (under the first $D^{-1/2}$ matrix)
- column-normalised (under the second $D^{-1/2}$ matrix)

II. What is GCN (Graph Convolution Network)

Needs for normalizing adjacency matrix

$\tilde{A}$: Normalized adjacency matrix

- GCN with $\tilde{A}$ normalized adjacency matrix

$$Z = \tilde{A}XW$$

0.25	0.25	0	0.25	0.25
0.25	0.25	0.25	0.25	0
0	0.5	0.5	0	0
0.25	0.25	0	0.25	0.25
0.33	0	0	0.33	0.33

$\tilde{A}$

×

65	150
64	220
55	300
60	250
62	190

Feature matrix

×

Weight matrix

=

Latent feature (Z)

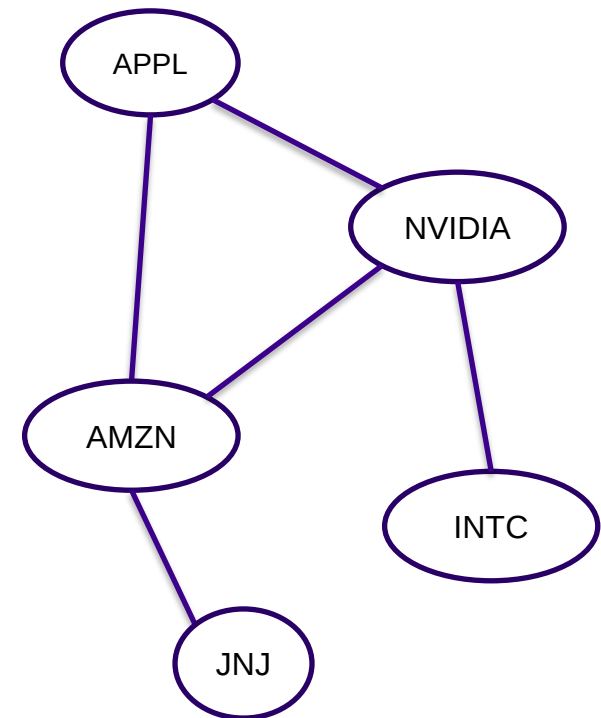
||| Experiment

III. Experiment

Application on our problem

Graph Representation

- $G = (V, E)$
 - G : Portfolio or ETF that holds various securities
 - V : Individual security
 - E : Connectivity based on prior domain knowledge
 - Correlation
 - Price, Return, Volatility
 - Causality
 - Granger, Transfer entropy
 - Corporate Governance
 - Etc



<Fig6. Stock security Graph example >

III. Experiment

Spatial Temporal (Spatio-temporal) Modelling

Spatial feature : GCN / Temporal feature : Temporal DL

- Spatial temporal data
Data is collected across both space and time.

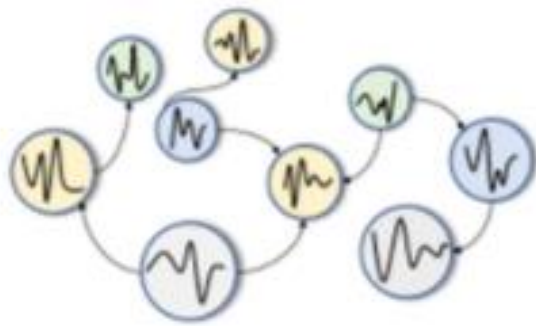


Figure 1

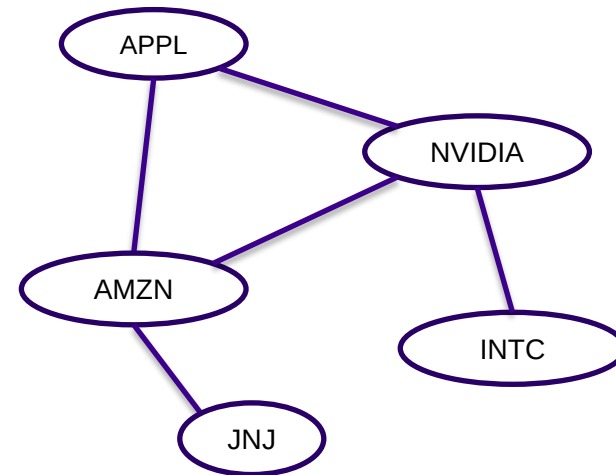
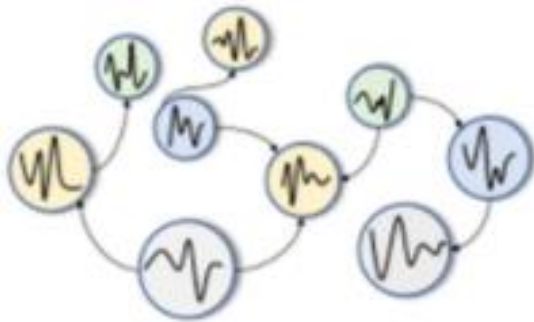
<Fig7. Spatio-Temporal data example: Traffic >

III. Experiment

Spatial Temporal (Spatio-temporal) Modelling

Spatial feature : GCN / Temporal feature : Temporal DL

- Securities in Portfolio/ETF

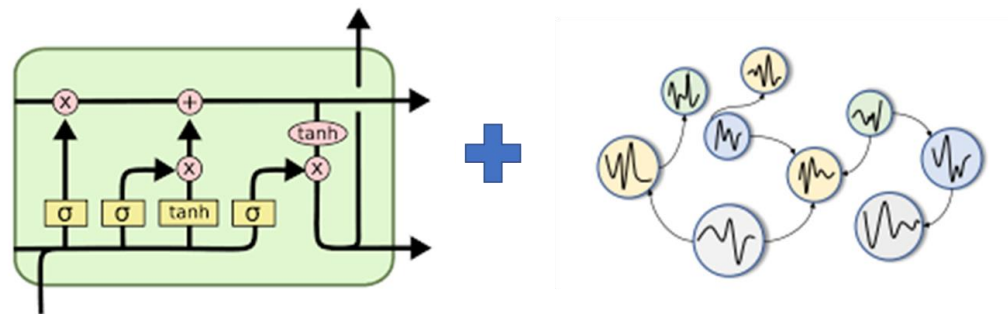


III. Experiment

Model Suggestion

LSTM + GCN

- Proposed Model : LSTM + GCN
 - Temporal learning : LSTM
 - Graph learning : GCN



<Fig8. Proposed Model >

III. Experiment

Proposed GCN Layer

$$z = \sum_{k=0}^K P^k X W_{k,1} + \tilde{A}_{apt} X W_{k,2}$$

- Proposed GCN Layer

$$z = \sum_{k=0}^K P^k X W_{k,1} + \tilde{A}_{apt}^k X W_{k,2}$$

- Diffusion convolution layer (P^k)

$$z = \sum_{k=0}^K P^k X W_{k,1}$$

- Self-adaptive adjacency layer ($\tilde{A}^k$)

$$z = \sum_{k=0}^K \tilde{A}_{apt}^k X W_{k,2}$$

III. Experiment

Diffusion GCN

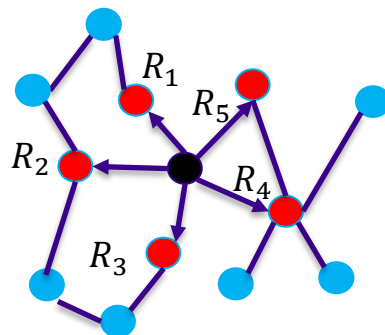
Modelling the **diffusion process** of graph signals with K finite steps.

- Diffusion convolution

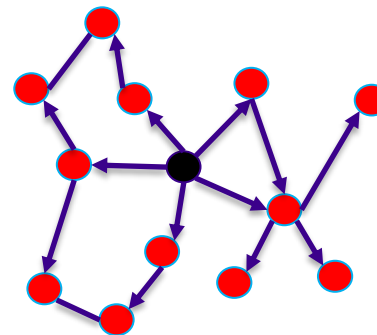
$$z = \sum_{k=0}^K P^k X W_{k,1}$$

P^k : power series of the transition matrix

k^{th} order = 1



k^{th} order = 2



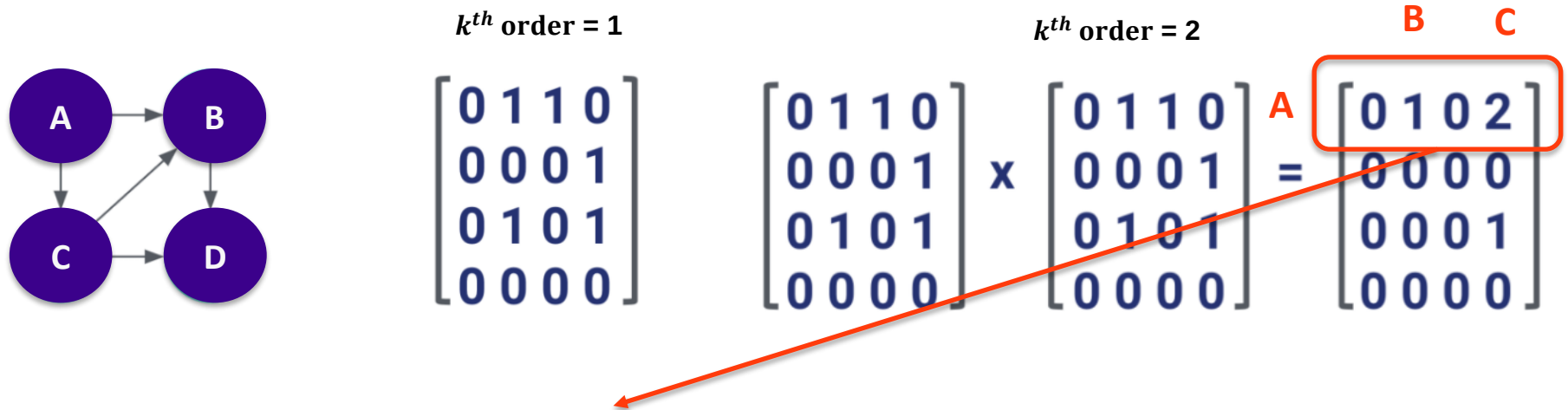
<Fig9. Diffusion Process >

III. Experiment

Diffusion Matrix

$$Z = \sum_{k=0}^K P^k X W_{k,1}$$

- Diffusion matrix P^k
 - $A^k / \text{rowsum}(A)$
: power series of the adjacency matrix



Number of path "A" to "B" with two steps

- [A, C, B]

Number of path "A" to "D" with two steps

- [A, C, D]
- [A, B, D]

III. Experiment

Self adaptive matrix

$$z = \sum_{k=0}^K \tilde{A}_{apt}^k X W_{k,2}$$

- Self adaptive matrix formula

$$\tilde{A}_{adp} = SoftMax(ReLU(E_1 E_2^T))$$

$$\tilde{A}_{adp} \in R^{N \times N}, E_1, E_2 \in R^{N \times c}, \text{ where } c \text{ is dimension for each } E_1, E_2$$

- To capture **hidden relationship** between nodes as the model train (stochastic gradient descent)
- Randomly initializing two nodes embeddings dictionaries with learnable parameters
- ReLU : to get ride of weak connections (negative value is not necessary in adjacency matrix)
 - $\text{Max}(0, z)$
- SoftMax : normalizing the matrix

III. Experiment

Self adaptive matrix

$$z = \sum_{k=0}^K \tilde{A}_{apt}^k X W_{k,2}$$

- Self adaptive matrix

```
tensor([[2.4922e-03, 4.7036e-05, 4.7036e-05, ..., 4.7036e-05, 4.7036e-05,
        4.7036e-05],
        [1.3714e-05, 1.7850e-05, 1.3714e-05, ..., 1.4060e-05, 1.3714e-05,
        1.3714e-05],
        [9.2193e-04, 7.7226e-05, 3.3764e-04, ..., 7.7226e-05, 7.7226e-05,
        7.7226e-05],
        ...,
        [1.0104e-05, 1.0104e-05, 3.6211e-03, ..., 1.0104e-05, 1.3565e-03,
        1.0104e-05],
        [1.1968e-04, 1.1968e-04, 1.1968e-04, ..., 3.3444e-04, 1.1968e-04,
        6.4537e-04],
        [1.2271e-03, 4.1015e-05, 1.0355e-04, ..., 2.0068e-04, 4.1015e-05,
        4.1015e-05]], device='cuda:0', grad_fn=<SoftmaxBackward>)
```

III. Experiment

Proposed GCN Layer

$$z = \sum_{k=0}^K P^k XW_{k,1} + \tilde{A}^k_{apt} XW_{k,2}$$

III. Experiment

Problem Statement

- Problem Statement

$$[X^{t-S:t}, G] \rightarrow f \rightarrow X^{t+1:t+T}$$

$$X^{t-S:t} \in R^{N \times D \times S}$$

$$X^{t+1:t+T} \in R^{N \times D \times T}$$

- Dataloader

train (shuffle = True)

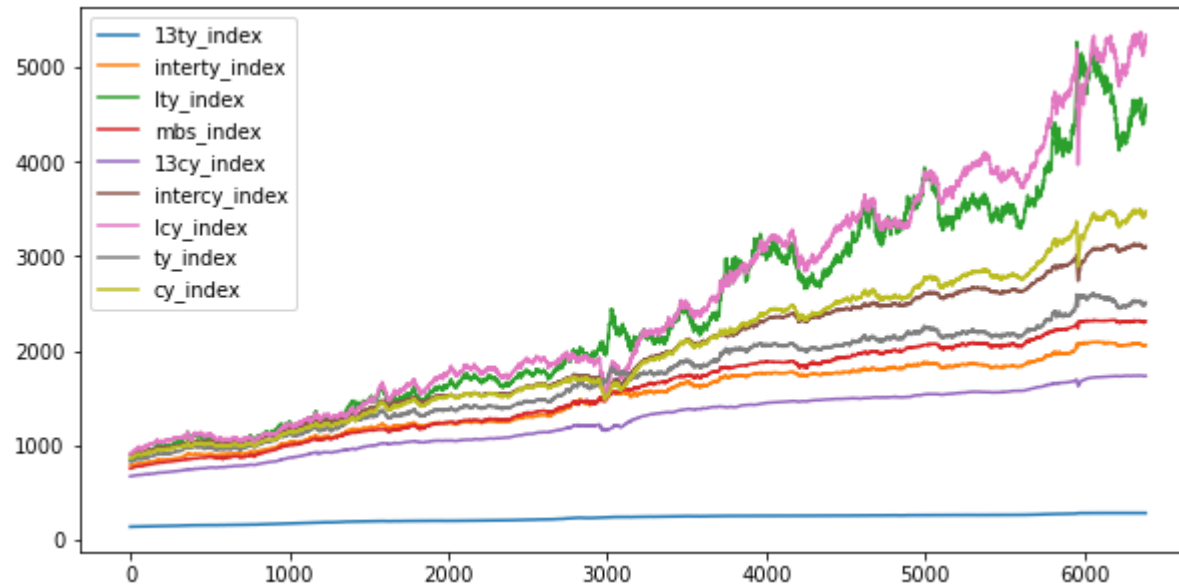
X : [3526, 66, 9, 1]

Y : [3526, 22, 9, 1]

III. Experiment

Data

Fixed Income data



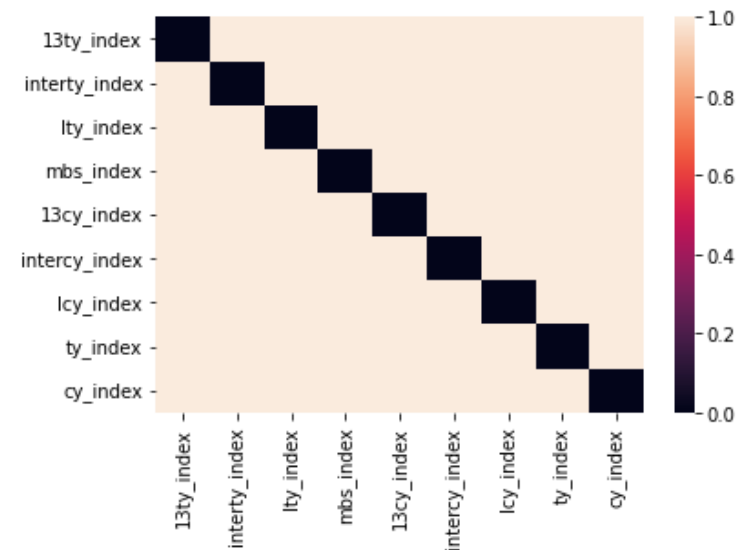
- Universe (9)
 - 13ty index', 'interty index', 'lty index', 'mbs index', '13cy index', 'intercy index', 'lcy index', 'ty index', 'cy index'

III. Experiment

Problem Setting

- Problem Setting
 - S : 66 days (3 past months)
 - T : 22 days (1 future month)
 - Train/Test : 7:3
- Customed adjacency matrix (**domain specific prior knowledge**)

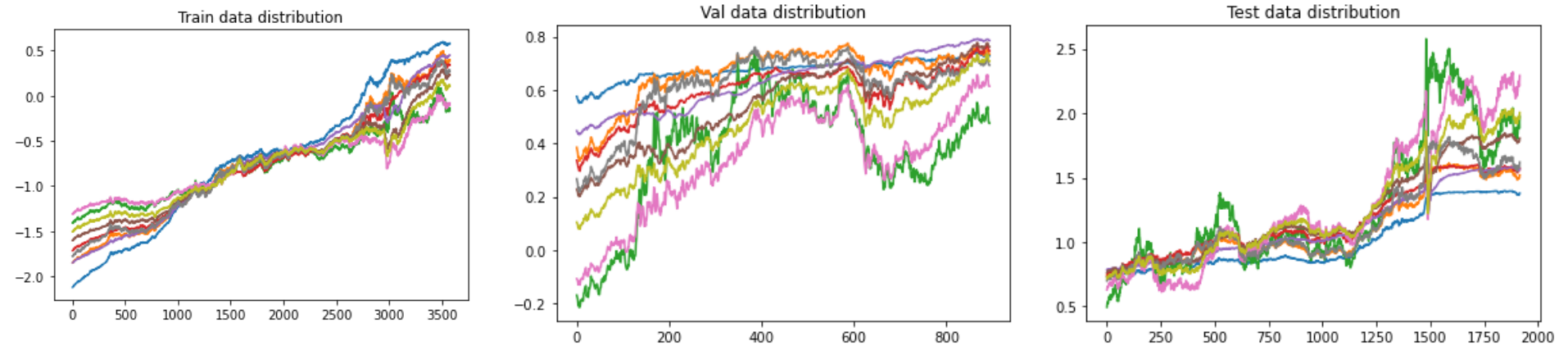
$A_{i,j} = 1$ if price correlation of v_i, v_j is above 0.7



III. Experiment

Data Distribution

Different data distribution



III. Experiment

Result

Loss : MAE

	1. LSTM (Test Loss: 450.259)	2. LSTM + GCN with only customized adj Test Loss : 293.529
Train Validation Loss	LSTM Loss(MAE) for Train/Val	GCN LSTM Loss(MAE) for Train/Val
	3. LSTM + GCN with only self adapt adj Test Loss: 259.204	4. LSTM + GCN with adj + self adj <u>Test Loss: 131.006</u>
Train Validation Loss	GCN LSTM Loss(MAE) for Train/Val	GCN LSTM Loss(MAE) for Train/Val

III. Experiment

Result

Self adaptive adj representation

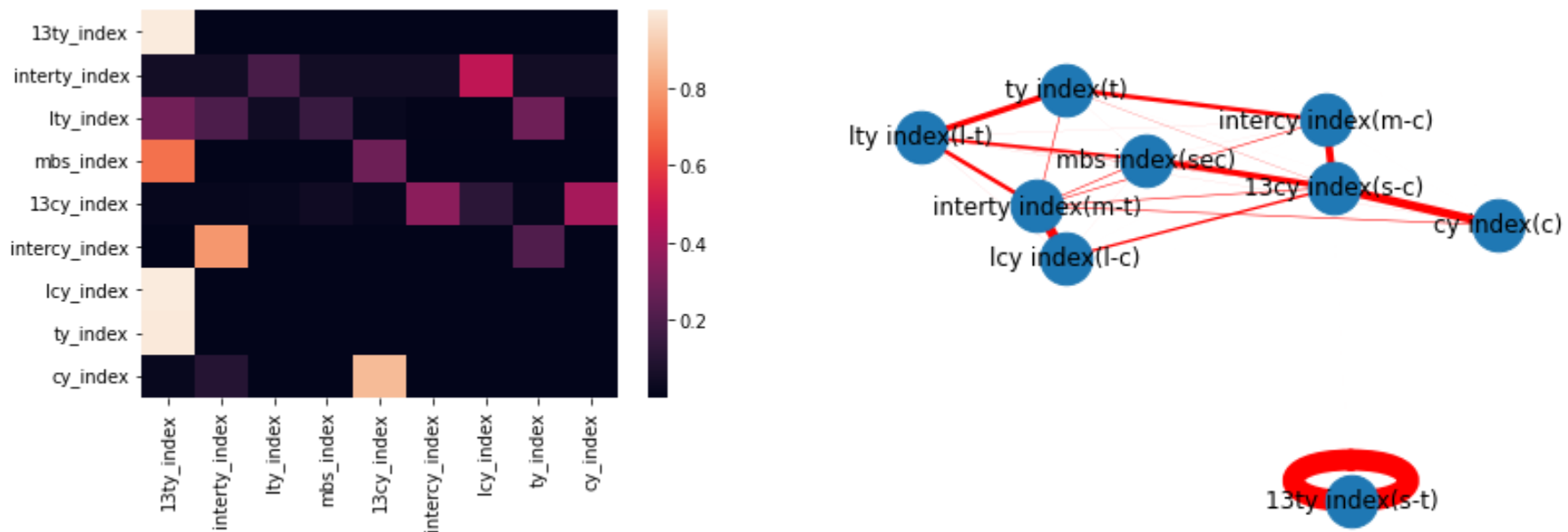
	3. LSTM + GCN with only self adapt adj Test Loss: 259.204	4. LSTM + GCN with adj + self adj Test Loss: 131.006
Train Validation Loss		
Self Adaptive adj		

III. Experiment

Result

Self adaptive adj visualization

- Self adaptive adj from 4. LSTM + GCN with adj + self adj



<Fig10. Generated graph information >

III. Experiment

Conclusion

- Plain Temporal model < Spatio-Temporal Model
- Domain Knowledge + Generated Information = Better Performance
- Additional Insight from the Graph Learning

IV Future work

IV. Future Work

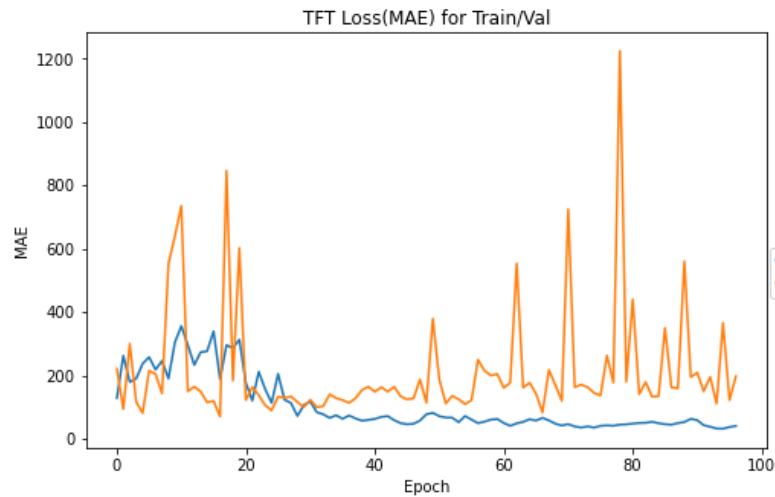
Concept & Task

- Graph Attention Network (GAT)
 - Detect attention value among nodes per t time step
- Node Classification
- Centrality Analysis
 - Node Importance

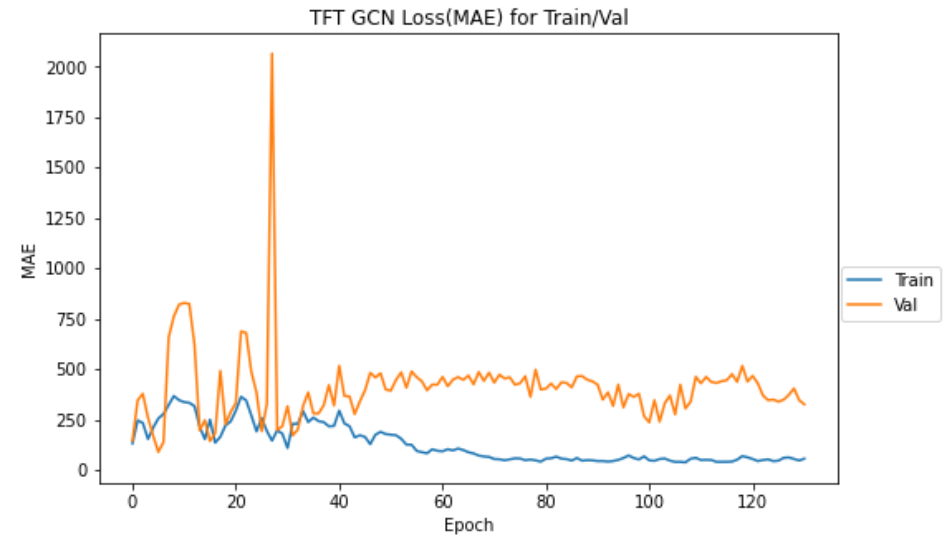
Appendix

TFT vs TFT with GCN

No difference : Due to lack of data compared to enormous number of model parameters



test loss 505.323



test loss 505.236

Reference

- [1] J.Baehr et. Al. “Machine Learning and Structural Characteristics of Reverse Engineering”
- [2] Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.
- [3] Shigehalli, V. S., & Shettar, V. M. (2011). Spectral techniques using normalized adjacency matrices for graph matching. International Journal of Computational Science and Mathematics, 3(4), 371-378.
- [4] Wu, Z., Pan, S., Long, G., Jiang, J., & Zhang, C. (2019). Graph wavenet for deep spatial-temporal graph modeling. arXiv preprint arXiv:1906.00121.
- [5] Sun, W., Tian, C., & Yang, G. (2015). Network analysis of the stock market.